4th April : Chai aur SQL

1 - Jaani

What is a DB?

A DB is a structured collection of data managed by a DBMS which allows efficient querying, updating, and maintaining data integrity.

DB doesn't store data in itself, but in the hardisk/SSD.

DB is only to get/write the data.

SQL VS NOSQL

⇒ RDBMS (Relational Data Base Management System)

- SQL is used with RDBMS
- ⇒ DBS who can understand SQL language are SQL dbs.
- It is built for durability, consistency, atomic transaction
- SQL in simpler form could be neat Excel sheets with strict rules
- SQL is highly structured, has predefined schemas and enforces ACID properties. This is optimized for reliance and durability.

  A - Atomicity
  C - Consistency
  I - Isolation
  D - Durability

⇒ NOSQL

- DBs who do not entertain SQL languages are NO-SQL dbs
- It is a type of non-relational database systems.
- It is designed for flexible, schema-less, or schema flexible.
- It is optimized for scalability and high performance.

ORM (Object Relational Mapper)

- is used with SQL
- It maps objects in code → tables in a relational DB

ODM (Object Document Mapper)

- used with ~~NOO~~ NOSQL ~~dbs~~ eg MongoDB)
- maps objects in code → documents (JSON like)

SQL (Structured Query Language)

01-class-sql

⇒ ~~cre~~ docker run -d --name sql-class-pg -e POSTGRES-PASSWORD=postgres -p 5433:5432 postgres:15

This cmd does ⇒

1. Starts a PostgresSQL db inside a container
2. Runs it in detached mode in background
3. Sets credentials via environment variables
4. Maps port, and starts a POSTGRES server.

docker exec -it sql-class-pg psql -U postgres

This cmd does ⇒

1. Finds running container sql-class-pg
2. Opens an interactive terminal inside it
3. Runs psql inside container

Now we can use

SELECT * FROM pg-database

If using VS code, install extension SQLTools

Create a table :

In SQL, use double quotes if you dont want SQL to convert it into lowercase.

3

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
```

→ integers, starts from 1 and auto increment it

→ unique, not null

```
    first_name VARCHAR(50) NOT NULL,
```

→ string, can be stored upto 50 characters

→ should not be null

```
    last_name VARCHAR(50),
```

← 50*8 bytes

```
    email VARCHAR(322) UNIQUE NOT NULL,
```

→ ensures no duplicate value exist in table

```
    phone_number VARCHAR(10) UNIQUE,
    country_code VARCHAR(4),
```

→ If you use number it takes 4 bytes that's too much.

```
    age INT CHECK (age >12),
```

→ ensures the column follows a condition.

```
    current_status VARCHAR(20) DEFAULT 'active'
    CHECK (current_status IN ('active', 'graduated', 'dropped out'),
    masterji_handle VARCHAR(50) UNIQUE,
    has_joined_masterji BOOLEAN DEFAULT FALSE,
```

↳ means this column's type is boolean and its default values is false.

```
    current_score INT DEFAULT 0 CHECK (current_score >=0 AND
    current_score <100),
    enrollment_date DATE DEFAULT CURRENT_DATE
)
```

↳ for storing dates.

```
ALTER TABLE students
  ADD COLUMN batch_name VARCHAR(50) DEFAULT
                        'Web Dev 2026';
```

Note: what is DDL (Data definition language)

It is a set of SQL commands used to define and modify the structure (schema) of a DB. eg CREATE, ALTER, DROP, TRUNCATE etc

In 02-class.sql file,

```
CREATE TABLE ipl_players (
    player_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    team VARCHAR(50),
    role VARCHAR(50),
    runs_scored INT CHECK(runs_scored >0),
    wickets_taken INT CHECK(wickets_taken >0),
    auction_price_crores ~~DECIMAL~~ INT
);
```

```
ALTER TABLE ipl_players
  ADD COLUMN nickname VARCHAR(50);
```

SELECT * FROM ipl_players [gets all colums & rows from a table]

SELECT name, nickname, team FROM ipl_players
↳ only fetch what you need.

To do filtering in table.

SELECT * FROM ipl-players WHERE team = 'Mumbai Indians';

↓ only select players who are from MI

SELECT name, nickname, auction-price-crores FROM ipl-player WHERE auction-price-crores > 10;

↳ only select players whose auction-prices-crores is greater than 10 crores.

⇒ **LOGICAL OPERATORS (AND, OR)**

SELECT * FROM ipl-players WHERE wickets_taken > 10 AND role = 'All-rounder'

↳ only players who took more than 10 wickets and role is "all-rounder"

SELECT * FROM ipl-player WHERE team = "CSK" OR team = 'RCB' ↓

Any is true give result.

⇒ **Pattern Matching**

SELECT * FROM ipl-player WHERE name LIKE '_ _ s %';

⇒ here _ represents one char, % any no of char

↓ third letter to be S

⇒ **LIKE VS ILIKE**

LIKE is case-sensitive

ILIKE is not case-sensitive, where 'jaani = Jaani

SELECT * FROM ipl-player WHERE team IN ('M.I', 'KKR', 'Gujrat Titans', 'CSK', 'RCB')

↳ to select players from only these teams.

SELECT * FROM ipl-player WHERE auction-price BETWEEN 10 and 15,

⇓ used to filter a range

SELECT * FROM ipl-players WHERE team <> 'CSK';

⇓ select rows where team is NOT equal to 'CSK'

⇒ Sorting

SELECT name, nickname ~~FROM~~, auction-price-crores
FROM ipl-players
ORDER BY auction-price-crores DESC.

↳ sorts players by auction price from highest to lowest

SELECT team, nickname, auction-price-crores
FROM ipl-players
ORDER BY team ASC, auction-price-crores DESC

↳ Sorts by team (A to Z)

↳ and within each team ~~the~~ shows ~~from~~ highest priced players first

Pagination

```
SELECT name, nickname, auction_price_crores
FROM ipl_players
ORDER BY auction_price_crores DESC
LIMIT 3;
```

⇒ This returns top 3 players with the highest auction price after sorting

```
SELECT name, nickname, auction_price_crores
FROM ipl_players
ORDER BY auction_price_crores DESC LIMIT 3 OFFSET 3
```

LIMIT 3 OFFSET (page_num − 1) * limit;

1. sort all players by price (highest → lowest)
2. Skips first 3 rows (OFFSET 3)
3. Returns next 3 rows (LIMIT 3)

Edge cases:

- LIMIT 3 OFFSET 1000 → gives empty result, if rows is less than 1000, so we can't skip 1000 rows, so no result.
- LIMIT 10 OFFSET 3 → skip first 3 rows, then give 10 rows (but if after skipping only 5 rows are left, only 5 are shown) (as no data is left to show).
- LIMIT 1000 OFFSET 0 → If table has also less than 1000 rows, the data which is present is shown, as nothing is skipped.

⇒ **Modifying data in runtime**

acts like a label

```
SELECT name, auction_price_crores,
(auction_price_crores * 1000) AS price_in_lakhs
FROM ipl_players;
```

↳ this is a temporary column name, in which we store the price converted from crores to lakhs.

⇒ SELECT
name, nickname, auction_price_crores(auction_price_crores + 2)
as new_price
FROM ipl_players;

↳ gives current price of each player and adds +2 to it, and stores it in a temp column named 'new_price'

Note: This does not change data in db, only for output purposes (temporary calculation)

⇒ **How you can get distinct values**

SELECT distinct role FROM ipl_players;

↳ only gets unique values from "role" column.
No duplicate value will be given.

DQL (Data Query Language)

⇒ Does only read / fetch in table
⇒ Does not update the table

* You can do WHERE, ORDER BY, GROUP BY, Read only operations allowed.

→ 03-class.sql

```
CREATE TABLE canteen_menu (
  item_id SERIAL PRIMARY KEY,
  item_name VARCHAR(100),
  category VARCHAR(50),
  price int,
  is_available BOOLEAN DEFAULT TRUE
)
```

// INSERT into canteen_menu

```
(item_name, category, price)
VALUES
('Masala Chai', 'Beverages', 10),
('Vada Pav', 'Snacks', 15);
```

} item_id not mentioned as will auto generate, bcs is_available not given, will take default 'true'

```
SELECT * FROM canteen_menu;
```

// Update the price of vada pav

```
→ UPDATE canteen_name
  SET price = 20
  WHERE item_name = 'Vada Pav';
```

} this is mutation, we assign the variable

// What if i want to decrease the price of beverages

```
→ UPDATE canteen_name
  SET price = price - 5
  WHERE category = 'Beverages';
```

→ reduce all beverage prices by 5

// What if Vada Pav is now unavable and also reduce price by 10 rupee

```
UPDATE canteen_~~name~~ menu
SET is_available = FALSE, price = 10
WHERE item_name = 'Vada Pav'
```

DELETE (don't run delete without where clause)


```
DELETE FROM canteen_menu
WHERE item_name = 'Vada Pav'
);
```

DML (Data Manipulation language)

⇒ We do changes in the table,
eg INSERT, UPDATE, DELETE

⇒ DML modify the data, but does not change table structure.

Note : DRY RUN in SQL, means first do SELECT query, then do DML queries.

Why important?
To avoid updating/deleting wrong data.

Note : TRUNCATE is a DDL query, WHERE clause is not used here. The auto-increment also resets here.

In 04-class.sql ⇒

Aggregate functions

1. COUNT()

```
SELECT COUNT(*) FROM smart_watch_sales
```

↳ counts including the duplicates

⇒ SUM() → gives the total value

SELECT SUM(units-sold * price-per-unit) AS total-revenue
FROM smart-watch-~~values~~ sales;

11

↳ For each row calculates
~~total~~ revenue = units_sold * price-per-unit
Then adds all rows together
give total_revenue = 50,000 etc
↓
random value

→ SELECT units-sold * price per-unit AS total-revenue
FROM smart-watch-sales; ⇒ not correct

→ SELECT AVG(price-per-unit) AS avg-price-per unit
FROM smart-watch-sales;
↳ total sum of prices/no of rows

→ SELECT MIN(price-per-unit) AS cheapest FROM smart-watch-sales
↳ Items that is cheapest

→ SELECT MAX(price-per unit) AS mehenga FROM smart-watch-sales
↳ items that is the costliest.

Group By

SELECT brand, SUM(units-sold) AS total-units-sold
FROM smart-watch-sales
GROUP BY brand
ORDER BY total-unit-sold DESC
↳ group rows by brand, sums units for each brand
then sorts the result from highest to lowest
that is descending order

```
SELECT
    city, SUM(units_sold * price_per_unit) as city_revenue
FROM smart_watch_sales
GROUP BY city;
```

↳ sorts cities by revenue, finds the highest revenue city, ~~then aggregates the city + brand~~ For each city, combines all brands inside that city.
At last, ~~ea~~ how much each city earned?

~~SELCE SELCE~~

```
SELECT city, brand
SUM(units_sold) As units
FROM smart_watch_sales
GROUP BY city, brand;
```

↳ In each city, each brand sells how many units.
eg, how much sales did Apple did in Mumbai etc.

Note: study about HAVING.